

Below is a complete student-facing instruction that you can post in Canvas, email to students, or include in the final project materials.

 Edit



GitHub Portfolio Final Project Instructions

Purpose

For this optional portfolio activity, you will publish your final project in a GitHub repository. A GitHub portfolio allows you to:

- Present your project to employers, instructors, and other professionals.
- Demonstrate your programming, data analysis, and documentation skills.
- Maintain a record of your project development.
- Share your work through a public web link.
- Continue improving your project after the course ends.

Publishing your project on GitHub does not replace the regular final project submission unless your instructor specifically states otherwise.

Part 1: Create a GitHub Account

Step 1.1: Open the GitHub Website

Go to:

<https://github.com>

Click **Sign up**.

Step 1.2: Register Your Account

Enter the requested information:

1. Email address
2. Password

3. GitHub username
4. Email preferences

Your username becomes part of your GitHub portfolio address.

For example:

`https://github.com/your-username`

Choose a professional username when possible. A combination of your first name, last name, initials, or professional field is recommended.

Examples:

`janedoe`
`jane-doe-data`
`jdoe-python`

Avoid usernames that may appear unprofessional when included in a résumé or job application.

Step 1.3: Verify Your Email Address

GitHub will send a verification message to your email account.

Open the message and complete the verification process before creating your repository.

Step 1.4: Secure Your Account

Create a strong password and enable two-factor authentication when possible.

Do not share your GitHub password, authentication token, or recovery codes with anyone.

Part 2: Create a GitHub Repository

A repository, often called a “repo,” is the online folder that stores your project files and development history.

Step 2.1: Start a New Repository

After signing in to GitHub:

1. Click the **+** icon in the upper-right corner.
2. Select **New repository**.
3. Enter a repository name.

Use a clear and professional repository name.

Examples:

```
python-data-analysis-final-project  
customer-churn-analysis  
housing-price-prediction  
uci-python-portfolio
```

Use hyphens or underscores instead of spaces.

Step 2.2: Add a Description

Write one sentence explaining the project.

Example:

```
An exploratory data analysis project examining California ho
```

Step 2.3: Choose Repository Visibility

Select one of the following:

- **Public:** Anyone can view the repository.
- **Private:** Only you and people you authorize can view the repository.

Choose **Public** when the project is ready to be included in your professional portfolio.

Before making the repository public, verify that it does not contain private, confidential, copyrighted, or personally identifiable information.

Step 2.4: Initialize the Repository

For the simplest setup, select:

- **Add a README file**
- A suitable `.gitignore`, such as **Python**, if available
- A license only when you understand which license is appropriate

Then click **Create repository**.

Important

If you plan to clone the repository immediately, initializing it with a README file is helpful because it ensures the repository already contains a branch and at least one file.

Part 3: Install the Required Software

You may use one of the following approaches:

1. Visual Studio Code with its integrated terminal
2. GitHub Desktop
3. Windows Command Prompt or PowerShell
4. macOS Terminal
5. Another Git-compatible development environment

Visual Studio Code is recommended because you can edit files and run Git commands in the same application.

Option A: Install Visual Studio Code

Download Visual Studio Code from the official website:

<https://code.visualstudio.com/download>

The official download page provides versions for Windows, macOS, and Linux.

After installation:

1. Open Visual Studio Code.
2. Select **File** → **Open Folder**.
3. Open your project folder.
4. Select **Terminal** → **New Terminal** to open the integrated terminal.

On Windows, restarting the terminal after installation may be necessary before commands such as `code` . become available.

Option B: Install GitHub Desktop

Download GitHub Desktop from:

<https://desktop.github.com/download/>

GitHub Desktop is available for Windows and macOS and provides a graphical interface for cloning repositories, committing changes, and pushing files.

This option may be easier for students who are not yet comfortable with command-line tools.

Option C: Install Git

Git must be installed before you can run Git commands from Command Prompt, PowerShell, Terminal, or Visual Studio Code.

Download Git from:

<https://git-scm.com/install/>

Select your operating system and follow the installation instructions. Git is a free, open-source version-control system.

Check Whether Git Is Installed

Open a terminal and enter:

```
git --version
```

A successful result will resemble:

```
git version 2.x.x
```

If the terminal reports that `git` is not recognized or cannot be found, install Git and restart your terminal or computer.

Part 4: Configure Git

Before making your first commit, tell Git which name and email address should appear in your project history.

Open Terminal, Command Prompt, PowerShell, or the Visual Studio Code terminal.

Enter:

```
git config --global user.name "Your Full Name"
```

Example:

```
git config --global user.name "Jane Doe"
```

Next, enter:

```
git config --global user.email "your-email@example.com"
```

Use an email address associated with your GitHub account.

Check the settings:

```
git config --global user.name  
git config --global user.email
```

You may also display all Git settings with:

```
git config --list
```

Part 5: Select a Location for Your Local Repository

Choose a location on your computer where you will store your GitHub projects.

Examples include:

Windows

```
C:\Users\YourName\Documents\GitHub
```

macOS

```
/Users/YourName/Documents/GitHub
```

You may create a folder named:

```
GitHub
```

or:

Projects

Avoid placing the repository inside another Git repository. Also avoid frequently moving or renaming the project folder after cloning it.

Part 6: Clone the GitHub Repository

Cloning downloads a copy of the online GitHub repository to your computer.

Step 6.1: Copy the Repository Address

Open your repository on GitHub.

1. Click the green **Code** button.
2. Select **HTTPS**.
3. Copy the repository address.

It should resemble:

```
https://github.com/your-username/your-repository.git
```

Step 6.2: Navigate to the Parent Folder

Windows PowerShell or Command Prompt

Example:

```
cd C:\Users\YourName\Documents\GitHub
```

macOS Terminal

Example:

```
cd ~/Documents/GitHub
```

You can use the following command to see your current location:

Windows

```
cd
```

macOS or Linux

```
pwd
```

You can list the files and folders in the current location with:

Windows

```
dir
```

macOS or Linux

```
ls
```

Step 6.3: Run the Clone Command

```
git clone https://github.com/your-username/your-repository.g
```

Replace the sample address with the address copied from your GitHub repository.

Example:

```
git clone https://github.com/janedoe/python-data-analysis-fi
```

Step 6.4: Enter the Repository Folder

```
cd your-repository
```

Example:

```
cd python-data-analysis-final-project
```

Step 6.5: Open the Repository in Visual Studio Code

Enter:

```
code .
```

The period means “open the current folder.”

If `code .` does not work, open Visual Studio Code manually and select:

```
File → Open Folder
```

Then select the cloned repository folder.

Part 7: Add Your Final Project Files

Copy your final project files into the cloned repository folder.

A Python portfolio project may include:

```
project-name/  
|  
├─ README.md  
├─ requirements.txt  
├─ .gitignore  
├─ LICENSE  
├─ data/  
|   └─ sample_data.csv  
├─ notebooks/  
|   └─ final_analysis.ipynb  
├─ src/  
|   └─ analysis.py  
├─ images/  
|   └─ project_visualization.png  
├─ reports/  
|   └─ final_report.pdf  
└─ tests/  
    └─ test_analysis.py
```

Your project structure does not have to match this example exactly. It should, however, be organized clearly enough that another person can understand it.

Files That Should Usually Be Included

Consider including:

- Python source files
- Jupyter notebooks
- A README file
- A requirements file
- Sample or approved datasets
- Charts and visualizations
- A final report
- Project documentation

- Testing files
 - Instructions for running the project
-

Files That Should Not Be Uploaded

Do not upload:

- Passwords
- Personal access tokens
- API keys
- Private SSH keys
- Student records
- Social Security numbers
- Medical information
- Confidential employer data
- Personally identifiable information
- Large temporary files
- Virtual environment folders
- System-generated files
- Proprietary datasets without permission

Never place authentication information directly inside a notebook or Python file.

Part 8: Create a `.gitignore` File

A `.gitignore` file tells Git which files and folders should not be uploaded.

For a Python project, the `.gitignore` file may include:

```
# Python cache files
__pycache__/
*.py[cod]

# Virtual environments
venv/
.env/
env/
```

```
# Environment variables and secrets
.env
*.env

# Jupyter temporary files
.ipynb_checkpoints/

# macOS system files
.DS_Store

# Windows system files
Thumbs.db

# Visual Studio Code settings
.vscode/

# Test and coverage files
.pytest_cache/
.coverage
htmlcov/

# Build files
build/
dist/
*.egg-info/
```

Important: If a password or token has already been committed, adding it to `.gitignore` does not remove it from the repository history. Revoke the exposed credential immediately and replace it.

Part 9: Review the Repository Status

Before adding files, check the repository status:

```
git status
```

The correct command is:

```
git status
```

Do not use:

```
git stats
```

`git status` shows:

- Your current branch
 - Files that have changed
 - Untracked files
 - Files prepared for a commit
 - Whether the local branch is ahead of or behind GitHub
-

Part 10: Check the Remote Repository

Run:

```
git remote -v
```

A result may resemble:

```
origin  https://github.com/your-username/your-repository.git
origin  https://github.com/your-username/your-repository.git
```

`origin` is the standard nickname for the GitHub repository connected to your local folder.

If no remote appears, add one with:

```
git remote add origin https://github.com/your-username/your-
```

Do not add another `origin` if one already exists.

To replace an incorrect remote address, enter:

```
git remote set-url origin https://github.com/your-username/y
```

Then check it again:

```
git remote -v
```

Part 11: Add Files to the Staging Area

After reviewing your changes, enter:

```
git add -A
```

This stages new, changed, renamed, and deleted files.

You may also use:

```
git add .
```

For this project, `git add -A` is recommended because it captures all tracked changes in the repository.

Run:

```
git status
```

The staged files should now appear under:

```
Changes to be committed
```

Part 12: Commit the Changes

A commit creates a saved checkpoint in the project history.

Enter:

```
git commit -m "Add initial final project files"
```

The text following `-m` is the commit message. It becomes part of the permanent development log.

Use a short, meaningful message describing what changed.

Good examples:

```
git commit -m "Add initial data analysis notebook"
```

```
git commit -m "Add customer churn visualizations"
```

```
git commit -m "Fix missing value preprocessing"
```

```
git commit -m "Update README installation instructions"
```

```
git commit -m "Add final report and presentation"
```

Avoid unclear commit messages such as:

```
git commit -m "stuff"
```

```
git commit -m "test"
```

```
git commit -m "changes"
```

```
git commit -m "final final"
```

A commit message should explain what was added, corrected, or improved.

Part 13: Authenticate with GitHub

GitHub no longer accepts an ordinary account password for Git operations over HTTPS. When authentication is required, use an approved authentication method such as GitHub CLI, Git Credential Manager, GitHub Desktop, SSH, or a personal access token.

Recommended Option 1: Sign In Through Visual Studio Code

When you push from Visual Studio Code:

1. Select the **Accounts** icon.
2. Choose **Sign in with GitHub**.
3. Your browser may open.
4. Sign in to GitHub.
5. Authorize Visual Studio Code.
6. Return to Visual Studio Code.

The application may store your authentication securely so you do not need to enter it for every push.

Recommended Option 2: Sign In Through GitHub Desktop

1. Open GitHub Desktop.
2. Select **File** → **Options** on Windows or **GitHub Desktop** → **Settings** on macOS.
3. Select **Accounts**.
4. Click **Sign in to GitHub.com**.
5. Complete authorization in your browser.

Recommended Option 3: GitHub CLI

GitHub documentation supports authentication through GitHub CLI for HTTPS or SSH connections.

After installing GitHub CLI, enter:

```
gh auth login
```

Follow the prompts and normally select:

```
GitHub.com
```

```
HTTPS
```

```
Login with a web browser
```

Option 4: Personal Access Token

A personal access token can be entered in place of a password for HTTPS Git operations. Treat the token like a password and never place it in your source code or repository.

To create one:

1. Sign in to GitHub.
2. Click your profile picture.
3. Select **Settings**.
4. Select **Developer settings**.
5. Select **Personal access tokens**.
6. Select **Fine-grained tokens** when appropriate.
7. Create a new token.
8. Limit access to the required repository.
9. Grant only the permissions needed.
10. Set an expiration date.
11. Generate and securely copy the token.

When Git asks for credentials:

```
Username: your-github-username
```

```
Password: paste-your-personal-access-token
```

The token is entered where Git displays the word “Password.” Do not enter your normal GitHub password.

Never share a personal access token with your instructor or classmates.

Part 14: Confirm the Branch Name

Check the current branch:

```
git branch --show-current
```

The result will usually be:

```
main
```

Some older repositories may use:

```
master
```

Use the branch name shown by the command. Do not assume that every repository uses the same branch name.

To rename the current branch to `main`, use:

```
git branch -M main
```

Part 15: Push the Project to GitHub

If the branch is named `main`, enter:

```
git push -u origin main
```

If the branch is named `master`, enter:

```
git push -u origin master
```

The correct word is:

```
origin
```

Do not type:

```
orgin
```

The `-u` option connects your local branch to the corresponding GitHub branch.

After this first successful push, you can usually use:

```
git push
```

Refresh the GitHub repository page in your browser. Your files should now appear online.

Part 16: The Complete First-Push Workflow

The following is the typical workflow after cloning a repository and adding project files:

```
cd path/to/your/repository
git status
git remote -v
git add -A
git status
git commit -m "Add initial final project files"
git branch --show-current
git push -u origin main
```

Use `master` instead of `main` only when `git branch --show-current` reports `master`.

Part 17: Repeat the Git Workflow as You Improve the Project

Continue working on your files normally.

Each time you complete a meaningful update, follow this cycle:

Step 17.1: Review Changes

```
git status
```

Step 17.2: Update Your Local Repository Before Pushing

When the online repository may have changed, run:

```
git pull
```

This is especially important when:

- You edited a file directly on GitHub.
- You are collaborating with another person.
- You are working from more than one computer.

- GitHub reports that your local branch is behind the remote branch.

Step 17.3: Stage the Changes

```
git add -A
```

Step 17.4: Review the Staged Files

```
git status
```

Step 17.5: Commit the Changes

```
git commit -m "Describe the completed update"
```

Example:

```
git commit -m "Add model evaluation results"
```

Step 17.6: Push the Changes

```
git push
```

Repeat this process until the repository is complete:

```
git pull
git status
git add -A
git status
git commit -m "Describe the update"
git push
```

Do not create a commit after every individual keystroke. Make a commit after completing a meaningful unit of work.

Part 18: Create a Professional README

The `README.md` file is normally the first page visitors see when they open your repository.

A strong README should include the following sections.

1. Project Title

```
# California Housing Price Analysis
```

2. Project Overview

Explain the project in one or two paragraphs.

Example:

This project analyzes California housing data to identify re

3. Project Objectives

Example:

```
## Objectives
```

- Clean and prepare the housing dataset.
- Explore important variables and distributions.
- Visualize relationships among housing characteristics.
- Develop a predictive model.
- Evaluate model performance.
- Communicate findings to a nontechnical audience.

4. Dataset

Include:

- Dataset name
- Source
- Number of observations
- Important variables
- Any usage restrictions

Do not upload a dataset unless you have permission to publish it.

5. Technologies Used

Example:

```
## Technologies Used
```

- Python
- pandas
- NumPy
- Matplotlib
- scikit-learn
- Jupyter Notebook

- Visual Studio Code
- Git and GitHub

6. Repository Structure

Example:

```
## Repository Structure

- `data/`: Sample or approved project data
- `notebooks/`: Jupyter notebooks
- `src/`: Python source code
- `images/`: Charts and screenshots
- `reports/`: Final project report
- `requirements.txt`: Python dependencies
```

7. Installation

Example:

```
## Installation
```

Clone the repository:

```
```bash
git clone https://github.com/your-username/your-repository.g
cd your-repository
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it on Windows:

```
.venv\Scripts\activate
```

Activate it on macOS or Linux:

```
source .venv/bin/activate
```

Install the required packages:

```
pip install -r requirements.txt
```

```
8. How to Run the Project
```

Example:

```
```markdown
## Running the Project
```

Start Jupyter Notebook:

```
```bash
jupyter notebook
```

Open the notebook located in the `notebooks` folder and run the cells in order.

```
9. Results
```

Summarize the most important findings.

Include selected charts when useful:

```
```markdown
![Project Visualization] (images/project_visualization.png)
```

10. Limitations

Explain important limitations involving:

- Data quality
- Sample size
- Missing values
- Modeling assumptions
- Generalizability
- Time limitations

11. Future Improvements

Describe how the project could be expanded.

12. Author

Example:

```
## Author
```

Jane Doe

Part 19: Create a `requirements.txt` File

The `requirements.txt` file identifies the Python packages needed to run the project.

Example:

```
pandas
numpy
matplotlib
scikit-learn
jupyter
```

You may create the file automatically with:

```
pip freeze > requirements.txt
```

However, `pip freeze` may include every package installed in your environment, including packages unrelated to the project. Review the file and remove unnecessary dependencies before publishing it.

A short, accurate requirements file is generally better than a long file containing unrelated packages.

Part 20: Test the Repository Before Sharing It

Before sharing your GitHub repository, test it from the perspective of another user.

Confirm that:

1. The repository opens successfully.
2. The README appears correctly.
3. All links work.
4. Images display correctly.
5. Required files are included.
6. Private information is not included.

7. The notebook runs from beginning to end.
8. Python scripts run without errors.
9. File paths are relative rather than tied to your computer.
10. The installation instructions are accurate.
11. The project results can be reproduced.
12. The repository has a meaningful commit history.

Avoid local file paths such as:

```
"C:/Users/Jane/Desktop/my_data.csv"
```

or:

```
"/Users/jane/Desktop/my_data.csv"
```

Use relative paths when possible:

```
"data/my_data.csv"
```

Part 21: Review the Git History

Display the project history with:

```
git log --oneline
```

Example:

```
8f25c13 Update README installation instructions
19ce842 Add model evaluation results
55e105a Add exploratory data visualizations
c8a420d Add initial final project files
```

A professional Git history demonstrates that the project developed through multiple meaningful stages.

Press  to exit the full Git log display when necessary.

Part 22: Make the Repository Ready to Share

Before sharing your repository with the GitHub community, complete the following review.

Repository Checklist

- ☐ The repository name is clear and professional.
 - ☐ The repository description explains the project.
 - ☐ The README is complete.
 - ☐ The project objective is clearly stated.
 - ☐ The dataset and source are documented.
 - ☐ The folder structure is organized.
 - ☐ The code contains useful comments.
 - ☐ The notebook runs in the correct order.
 - ☐ The requirements file is included.
 - ☐ Installation instructions are provided.
 - ☐ Project findings are summarized.
 - ☐ Important visualizations are included.
 - ☐ Limitations are explained.
 - ☐ Future improvements are described.
 - ☐ No passwords, tokens, or API keys are included.
 - ☐ No private or confidential information is included.
 - ☐ Large and unnecessary files are removed.
 - ☐ The `.gitignore` file is included.
 - ☐ Commit messages are clear.
 - ☐ All final changes have been pushed.
 - ☐ The GitHub repository link works in a private browser window.
-

Part 23: Share the Repository

Open the repository on GitHub and copy the browser address.

It should resemble:

`https://github.com/your-username/your-repository`

You may share this link through:

- Your résumé
- LinkedIn
- A personal portfolio website
- A job application
- An internship application
- An email to an instructor
- A course discussion board

- A professional networking profile

Example résumé entry:

California Housing Price Analysis | Python, pandas, scikit-learn
<https://github.com/your-username/california-housing-analysis>

Before sharing the repository publicly, open the link in a private or incognito browser window. This confirms that other people can access it.

Optional: Add the Repository to Your GitHub Profile

You may pin the project so that it appears near the top of your GitHub profile.

1. Open your GitHub profile.
2. Find the **Popular repositories** section.
3. Select **Customize your pins**.
4. Select the final project repository.
5. Save the changes.

Choose your strongest and most complete projects for your pinned repositories.

Common Git Problems and Solutions

Problem 1: `git` Is Not Recognized

Example:

```
'git' is not recognized as an internal or external command
```

Solution:

1. Install Git from <https://git-scm.com/install/>.
2. Restart the terminal.
3. Run:

```
git --version
```

Problem 2: `fatal: not a git repository`

Cause: You are not currently inside the cloned repository folder.

Check your current location:

```
pwd
```

or on Windows:

```
cd
```

List the folders:

```
ls
```

or:

```
dir
```

Enter the repository folder:

```
cd your-repository
```

Then run:

```
git status
```

Problem 3: `remote origin already exists`

Cause: The repository already has a remote named `origin`.

Check it:

```
git remote -v
```

To correct the address:

```
git remote set-url origin https://github.com/your-username/y
```

Problem 4: Authentication Failed

Possible causes include:

- Using your GitHub password instead of a token
- An expired token
- Incorrect stored credentials
- Insufficient repository permission
- Signing in with the wrong GitHub account

GitHub recommends credential helpers, GitHub CLI, or other approved authentication methods for HTTPS repositories.

Possible solutions:

```
gh auth login
```

Or sign out and sign in again through Visual Studio Code or GitHub Desktop.

When Git explicitly requests a password for an HTTPS operation, use a personal access token rather than your normal GitHub password.

Problem 5: `src refspec main does not match any`

Possible causes:

- No commit has been created.
- The branch is not named `main`.

Check the status:

```
git status
```

Create a commit:

```
git add -A
git commit -m "Add initial project files"
```

Check the branch:

```
git branch --show-current
```

Push the branch that actually exists:

```
git push -u origin main
```

or:

```
git push -u origin master
```

Problem 6: Push Rejected Because the Remote Contains Work

Example:

Updates were rejected because the remote contains work that

Pull the online changes first:

```
git pull --rebase origin main
```

Then push:

```
git push origin main
```

Use `master` instead of `main` when appropriate.

If Git reports a conflict, resolve the conflicting files before continuing.

Do not use a force push unless you understand its effect on repository history.

Problem 7: Nothing to Commit

Example:

```
nothing to commit, working tree clean
```

This usually means:

- No files have changed since the previous commit.
- The files were saved outside the repository folder.
- The changes have already been committed.

Run:

```
git status
```

Confirm that your files are inside the correct cloned repository folder.

Problem 8: A File Is Too Large

GitHub may reject very large files.

Possible solutions:

- Remove unnecessary files.
- Compress the output when appropriate.
- Upload only a small sample dataset.
- Add large generated files to `.gitignore`.
- Link to an authorized external data source.
- Use Git Large File Storage when appropriate.

Do not upload large files simply because they exist in your project folder.

Problem 9: Accidentally Uploaded a Secret

Immediately:

1. Revoke or rotate the exposed password, token, or API key.
2. Remove it from the project files.
3. Add the relevant file to `.gitignore`.
4. Commit and push the correction.
5. Review the repository history.

Deleting the secret from the newest version does not automatically remove it from earlier commits. Assume an exposed credential is compromised and replace it immediately.

GitHub Desktop Workflow

Students who prefer not to use terminal commands may complete the project through GitHub Desktop.

Clone the Repository

1. Open GitHub Desktop.
2. Sign in to GitHub.
3. Select **File** → **Clone Repository**.
4. Select the repository.

5. Choose a local folder.
6. Click **Clone**.

Add Project Files

Copy your final project files into the cloned folder.

Commit the Changes

1. Return to GitHub Desktop.
2. Review the changed files.
3. Enter a summary such as:

```
Add initial final project files
```

4. Click **Commit to main**.

Push the Changes

Click:

```
Push origin
```

Continue Updating

Each time you make changes:

1. Save the files.
2. Review the changes in GitHub Desktop.
3. Write a meaningful commit message.
4. Commit the changes.
5. Click **Push origin**.

Final Command Reference

First-Time Configuration

```
git config --global user.name "Your Full Name"  
git config --global user.email "your-email@example.com"
```

Clone a Repository

```
git clone https://github.com/your-username/your-repository.g  
cd your-repository
```

Check the Repository

```
git status
git remote -v
git branch --show-current
```

Add, Commit, and Push

```
git add -A
git status
git commit -m "Describe the completed update"
git push -u origin main
```

Later Updates

```
git pull
git status
git add -A
git commit -m "Describe the completed update"
git push
```

View Commit History

```
git log --oneline
```

Final Submission Instructions

Submit the following items if requested by your instructor:

1. Your full name
2. Project title
3. GitHub username
4. Public repository link
5. A brief project description
6. Any special instructions required to run the project

Example:

Student: Jane Doe

Project: California Housing Price Analysis

GitHub Username: janedoe

Repository: <https://github.com/janedoe/california-housing-an>

Description:

This project uses Python, pandas, data visualization, and re

Your repository should represent your own work and should clearly identify any outside datasets, libraries, tutorials, code, or other resources used in the project.

This version corrects the command errors in the original outline, including `git status`, `origin`, and the distinction between the `main` and `master` branches.